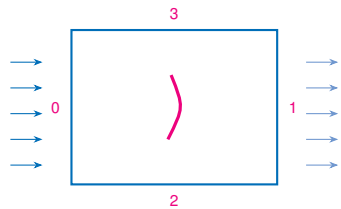
BOUNDARY CONDITIONS AND DOMAIN SET-UP

What happens at the edges of the box, how to say it in the input file, and how to size the box and its grid

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part III • Chapter 7



$a\,u + b\,\partial u/\partial n = g$ on every side

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

1. write down a Robin boundary condition, say what the coefficients a, b and g mean, and say which pairs (a, b) IBAMR's fluid solver actually accepts (only two);
2. read and write the `VelocityBcCoefs_0`, `_1`, `_2` blocks of an input file: the side numbering, the three keys per side, and the expressions in the quotes (variables, constants, functions, conditions);
3. set up inflow, outflow, walls, moving walls, open sides and periodic sides, time-dependent and space-dependent conditions, and initial conditions, and know what IBAMR does and does not let you say about the pressure;
4. size a rectangular domain and its grid: `domain_boxes`, `x_lo`, `x_up`, levels, refinement ratio, coarsest and finest cell counts, patch sizes, and the divisibility rules that go with them;
5. recognize the errors and warnings IBAMR prints for a broken boundary block, and the mistakes it accepts silently.

HOW THIS CHAPTER WAS CHECKED

- Every statement about what IBAMR does was checked against the 0.19.0 source: `INSStaggeredVelocityBcCoef.cpp` and `INSStaggeredPressureBcCoef.cpp` (what the coefficients do to the velocity and the pressure), `muParserRobinBcCoefs.cpp` (how the quoted expressions are read and which names they may use), `StaggeredStokesOpenBoundaryStabilizer.cpp`, `INSHierarchyIntegrator.cpp` and `AppInitializer.cpp` (the keys and warnings quoted).
- The error and warning messages are from short runs of IB/03 with deliberately broken boundary blocks (slides 66–67), IBAMR 0.19.0 on one process; the script that made them is in the references.
- The figures are from runs of IB/03 and IB/02 in this series with their shipped inputs, rendered in VisIt 3.5.
- Where the 2013–2020 lectures said something else, the slide says so, and slides 73–74 list the changes.

> Slide numbers on the slides refer to this deck. The chapter numbers (5, 6, 8a, 11, 15, 16) refer to the other chapters of Part III.

WHERE TO LOOK THINGS UP

- The examples: every example in this series has its boundary and geometry blocks commented in `input2d` or `input3d`, and its README says what the conditions do. Start from the one closest to what you want (slides 31–40).
- The source: `src/navier_stokes/INSStaggeredVelocityBcCoef.cpp` (what a, b, g do to the velocity), `INSStaggeredPressureBcCoef.cpp` (what they do to the pressure), `ibtk/src/boundary/physical_boundary/muParserRobinBcCoefs.cpp` (how the quoted expressions are read), all unpacked by autoibamr under `tmp/unpack/IBAMR-<version>/`.
- muParser, the expression parser, has its own manual: https://beltoforion.de/en/muparser/ (IBAMR bundles version 2 in `ibtk/contrib/muparser`).
- The grid blocks (`CartesianGeometry`, `GriddingAlgorithm`) belong to SAMRAI, which IBAMR bundles as IBSAMRAI2; the comments in the examples' input files are the practical reference.
- The IBAMR FAQ on the wiki, https://github.com/IBAMR/IBAMR/wiki/FAQ, and the users' group, https://groups.google.com/g/ibamr-users, where the pressure questions on slides 15–17 were answered.

THE BLOCKS OF THE INPUT FILE THIS CHAPTER COVERS

Block	What it sets	Slides
`VelocityBcCoefs_0`, `_1`, (`_2`)	the boundary condition of one velocity component on every side of the box	19–44
`VelocityInitialConditions`, `PressureInitialConditions`	the velocity (and, for output only, the pressure) at $t = 0$	47–48
`CartesianGeometry`	the index box of the coarsest grid, the physical corners of the domain, and which directions are periodic	45, 50
`GriddingAlgorithm`	the number of levels, the refinement ratio, the largest and smallest patch, the efficiency settings	51–58
`StandardTagAndInitialize` and the tagging keys of the integrator block	which cells are refined: gradient detector or vorticity thresholds, the tag buffer, how often to regrid	58–60
`IBHierarchyIntegrator` time keys	`dt_max`, `end_time`, `max_integrator_steps`, `grow_dt`	62–64
the output keys (`viz_dump_interval`, `data_dump_interval`)	how often frames and data files are written, in time steps	63

The physical constants at the top of the file (`MU`, `RHO`, the stiffnesses) are chapter 8a; the structure blocks (`IBStandardInitializer`) are chapter 6; the solver keys (`DELTA_FUNCTION`, `CFL_MAX`, `DT`) are chapter 5.

Part I: boundary conditions

Robin conditions and what IBAMR makes of them; the side numbering; the `VelocityBcCoefs` blocks; the expressions in the quotes; walls, inflow, outflow, open and periodic sides; time- and space-dependent conditions; the examples; the pressure; initial conditions.

Part II (from slide 49) sizes the box and the grid.

ROBIN BOUNDARY CONDITIONS

Robin boundary conditions are a weighted combination of Dirichlet (value) and Neumann (derivative) conditions. If Ω is the domain on which the equation is solved and $\partial\Omega$ its boundary, the Robin condition is

$$a\,u + b\,\frac{\partial u}{\partial n} = g \quad \text{on } \partial\Omega,$$

for constants a and b, not both zero, and a given function g on $\partial\Omega$. Here $\partial u/\partial n = \nabla u \cdot \mathbf{n}$ is the derivative along the outward unit normal $\mathbf{n}$.

- $a = 1$, $b = 0$: a Dirichlet condition, $u = g$: the value is prescribed.
- $a = 0$, $b = 1$: a Neumann condition, $\partial u/\partial n = g$: the outward derivative is prescribed.
- both nonzero: a true Robin (mixed) condition, used for scalars (slide 40) but not for the velocity (slide 10).

> IBAMR asks for a, b and g separately for every side of the box and every velocity component, each as an expression in quotes. That is all a `VelocityBcCoefs` block is: $2\,\mathrm{NDIM}$ sides $\times$ three expressions.

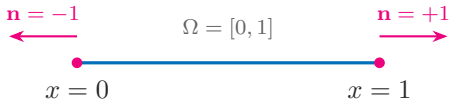
EXAMPLE: ROBIN CONDITIONS IN ONE DIMENSION

In one dimension, with $\Omega = [0, 1]$, the Robin condition becomes two conditions, one per end:

$$a\,u(0) - b\,u'(0) = g_0, \qquad a\,u(1) + b\,u'(1) = g_1.$$

Notice the change of sign in front of the derivative term. The outward normal at $x = 0$ points in the negative direction, so $\partial u/\partial n = -u'(0)$ there, while at $x = 1$ it points in the positive direction and $\partial u/\partial n = +u'(1)$.

The same happens on every side of a box: on a lower side (left, bottom, back) the outward normal is $-\mathbf{e}_d$, on an upper side it is $+\mathbf{e}_d$. IBAMR keeps track of this for you; you give a, b, g and never type a minus sign for the normal.



NO SLIP AND NEUMANN IN THIS NOTATION, AND A CORRECTION

- No-slip walls (Dirichlet): $a = 1$, $b = 0$, $g = 0$ gives $u(0) = 0$ and $u(1) = 0$.
- Neumann: $a = 0$, $b = 1$, $g = 0$ gives $\partial u/\partial n = 0$ at both ends, that is $-u'(0) = 0$ and $u'(1) = 0$.

> A correction to the 2013–2020 slides. They wrote the Neumann case as "$a = 0$, $b = -1, 1$, $g = 0$", with $b = -1$ at $x = 0$ and $b = +1$ at $x = 1$. That is the same condition written with u' instead of $\partial u/\partial n$, and it is correct as mathematics. It is wrong as an input file: IBAMR's condition is written with the outward normal derivative, so $b = 1$ on *both* ends, and the solver accepts only $b = 1$ (or $b = 0$). Typing -1 is an error (slide 10, tested on slide 66).

Dirichlet and Neumann are the only two cases the fluid solver takes, and they mean rather more than "value" and "derivative" for a velocity component: slide 11.

WHAT IBAMR DOES WITH a, b AND g: ONLY TWO CHOICES

For the velocity the coefficients are not used as a general Robin condition. The class that interprets them (`INSStaggeredVelocityBcCoef`) tests only whether $a = 1$ or $b = 1$:

```
const bool velocity_bc = IBTK::rel_equal_eps(alpha, 1.0);   // a == 1 : a velocity condition
const bool traction_bc = IBTK::rel_equal_eps(beta, 1.0);    // b == 1 : a traction condition
TBOX_ASSERT((velocity_bc || traction_bc) && !(velocity_bc && traction_bc));   // debug builds only
```

- $(a, b) = (1, 0)$: velocity condition, $u_d = g$ on that side (an inflow profile, a wall, a moving wall).
- $(a, b) = (0, 1)$: traction condition, a condition on the stress rather than the velocity (an open side, a free-slip wall); what exactly depends on whether d is the normal or a tangential component (slides 11, 13).
- Anything else (both 1, both 0, 0.5 and 0.5, -1): a release build stops with `this statement should not be reached!` from `INSProjectionBcCoef.cpp`, a debug build with the assertion above. Tested on slide 66.

> So g is a velocity when $a = 1$ and a *stress* (force per area, units of pressure) when $b = 1$. The same letter, two different physical quantities.

THE FOUR CONDITIONS A VELOCITY COMPONENT CAN CARRY

(a, b)	component	condition imposed	use it for
$(1, 0)$	normal	$u_n = g$: the flow through the side is prescribed	inflow and outflow profiles; a wall ($g = 0$); a pulsing inlet
$(1, 0)$	tangential	$u_t = g$: the velocity along the side is prescribed	no slip ($g = 0$); a sliding lid or belt ($g = V$)
$(0, 1)$	normal	`TRACTION`: $-p + 2\mu\,\partial u_n/\partial n = g$; `PSEUDO_TRACTION`: $-p = g$	an open side where the flow chooses its own velocity; sets the pressure level there
$(0, 1)$	tangential	`TRACTION`: $\mu(\partial u_t/\partial n + \partial u_n/\partial t) = g$; `PSEUDO_TRACTION`: $\mu\,\partial u_t/\partial n = g$	a free-slip (shear-free) wall or open side with $g = 0$; a prescribed shear stress

"Normal" means the component along the axis the side faces: u_0 (x) on sides 0 and 1, u_1 (y) on sides 2 and 3, u_2 (z) on sides 4 and 5 (numbering on slide 18). The other components are tangential there. For the normal component with $(0, 1)$ the velocity itself gets $\partial u_n/\partial n = 0$ (the source sets $\alpha = 0$, $\beta = 1$, $\gamma = 0$ for it) and g goes into the pressure condition instead (slide 14).

The choice between `TRACTION` (the default) and `PSEUDO_TRACTION` is the key `traction_bc_type` in the `INSStaggeredHierarchyIntegrator` block (slide 13).

VELOCITY CONDITIONS: WALLS, INFLOW, MOVING WALLS

With $(a, b) = (1, 0)$ on a side, g is the velocity component there.

- A fixed wall: $g = 0$ for all components on that side. The normal component 0 means nothing flows through; the tangential components 0 mean no slip.
- An inflow: the normal component $g = U$ (or a profile in the tangential coordinate, slide 29, or a ramp in time, slide 28); the tangential components 0.
- A moving wall (lid, belt, sliding floor): the normal component 0, a tangential component $g = V$. IB/08's lid (slide 38) and the top and bottom of IB/03's box (slide 21) are moving walls.
- An outflow with a prescribed velocity: the same as an inflow with the sign reversed. IB/03 and IBFE/10 prescribe the velocity on every side, including the outlet: a wind tunnel whose walls move with the stream.

> If the normal velocity is prescribed on *every* side (no open side, nothing periodic), the prescribed flux must balance: $\oint_{\partial\Omega} u_n \, dS = 0$, or the incompressibility constraint cannot be met. A uniform U on all sides balances; a parabola in and the same parabola out balances; an inlet with closed walls everywhere else does not. Slide 41 has the pressure side of this.

TRACTION CONDITIONS: OPEN SIDES

With $(a, b) = (0, 1)$ on a side the velocity is not prescribed; the stress is. The two forms:

- `TRACTION` (the default): the physical traction $\boldsymbol{\sigma}\mathbf{n}$ with $\boldsymbol{\sigma} = -p\mathbf{I} + \mu(\nabla\mathbf{u} + \nabla\mathbf{u}^T)$. Normal component: $-p + 2\mu\,\partial u_n/\partial n = g$. Tangential: $\mu(\partial u_t/\partial n + \partial u_n/\partial t) = g$.
- `PSEUDO_TRACTION`: drops the transposed gradient. Normal: $-p = g$. Tangential: $\mu\,\partial u_t/\partial n = g$.

Choose with a key in the fluid integrator's block (any of the integrators; the default is `TRACTION`):

```
INSStaggeredHierarchyIntegrator {
   ...
   traction_bc_type = "PSEUDO_TRACTION"   // or "TRACTION" (default)
}
```

- $g = 0$ on all components of a side is the usual open (outflow) boundary: zero normal stress, zero shear. The flow leaves (or enters) at whatever velocity the interior dictates, and the pressure there is $2\mu\,\partial u_n/\partial n \approx 0$.
- $g = 0$ on the tangential components only, with $u_n = 0$ prescribed, is a free-slip wall.

HOW THE PRESSURE GETS ITS BOUNDARY CONDITION

The pressure is not a state variable and has no block of its own. Its boundary condition is derived from the *normal* velocity component's condition on each side (`INSStaggeredPressureBcCoef.cpp`):

```
// Normal velocity boundary conditions are converted into Neumann conditions for the pressure, and
// normal traction boundary conditions are converted into Dirichlet conditions for the pressure.
case TRACTION:         // -p + 2*mu*du_n/dx_n = g.
    // p^{n+1/2} = mu*du_n/dx_n^{n} + mu*du_n/dx_n^{n+1} - g^{n+1/2}.
case PSEUDO_TRACTION: // -p = g.
```

- A velocity condition on the normal component gives a homogeneous Neumann condition for the pressure: the projection does not change the normal velocity at the boundary.
- A traction condition on the normal component gives a Dirichlet condition: the pressure at the boundary is $-g$ (pseudo-traction) or $2\mu\,\partial u_n/\partial n - g$ (traction, evaluated half at the old and half at the new time).

> So an outlet pressure p_{out} is imposed by $a = 0$, $b = 1$, $g = -p_{\text{out}}$ on the normal component of that side (and $g = 0$ on the tangential ones). Note the sign: g is the normal traction, $-p$.

“CAN I SPECIFY THE PRESSURE?”: THEN AND NOW

The 2013 lecture collected “random notes” on this question, because at the time the answer in the examples was no:

- From Boyce Griffith, quoted on the 2013 slide: “You cannot specify the pressure — pressure is computed by the fluid solver — but you can specify (in the code) a relationship between pressure and source/sink strength. You have to do all of this in the code — i.e. there are not input parameters to set source strengths.”
- So pressure-driven flow was done with sources and sinks inside the domain (a model of the reservoirs), coupled to the computed pressure (slide 16), or by prescribing the velocity.

> Now: the traction condition of slides 13–14 lets you set the pressure *on a side of the box* directly from the input file, with $(a, b) = (0, 1)$ and $g = -p$ on the normal component. A pressure drop between an inlet and an outlet is two such sides with different g; the flow rate is then an output. Sources and sinks are still there (the `.source` file and `IBStandardSourceGen`, chapter 6) for reservoirs that are inside the box.

Two cautions: an open inlet with traction conditions can admit flow in either direction, and an open outlet with backflow can go unstable; slide 44 has the stabilizer for that. And with an open side the pressure level is fixed by the boundary, so `normalize_pressure` must be `FALSE` (slide 41).

BOYCE'S HINTS ON RELATING PRESSURE AND SOURCE STRENGTH (2013)

Kept from the 2013 slide, because the reasoning still applies to any reservoir model you build in code. The question was how to pick the strength of a source for an inlet at pressure 2000 and two outlets at pressure 0.

- The idea is a relationship $Q_{\mathrm{src}} = f(Q_{\mathrm{src}},\, P_{\mathrm{rsvr}} - P_{\mathrm{src}})$, where Q_{src} is the source/sink strength, P_{src} the pressure measured at the source (computed by the fluid solver) and P_{rsvr} the reservoir pressure specified as the external condition.
- A simple resistive connection: $Q_{\mathrm{src}} = (P_{\mathrm{rsvr}} - P_{\mathrm{src}})/R$. With Q_{src} counted as flow *into* the domain (how IBAMR sets it up), the signs give inflow when $P_{\mathrm{src}} < P_{\mathrm{rsvr}}$.
- In practice a bit of delay is needed for stability, by adding inertance: $R\,Q_{\mathrm{src}} + L\,\dot{Q}_{\mathrm{src}} = P_{\mathrm{rsvr}} - P_{\mathrm{src}}$. His guess for why: the interior is coupled to the reservoirs explicitly in time; an implicit coupling could use a purely resistive connection.

Flow meters and pressure gauges were built out of "rings" of immersed-boundary points: the flow is the total velocity flux through a web spanning the ring, and the pressure is read at the ring's centroid. That is still how the `.inst` file works (chapter 6, `IBInstrumentPanel`).

THE SAME NOTE: INITIAL CONDITIONS, AND WHERE THE EXAMPLE IS

Also quoted on the 2013 slide, from the users' group, about a non-uniform incoming velocity profile ("linear increasing in z-direction") as the initial state:

> *"I think you are asking about setting up initial conditions. It is pretty straightforward to do this; see IBAMR/examples/navier_stokes/ex0 for an example. There are several different input files that set up nontrivial initial conditions for the velocity and pressure. (Note that the initial condition for the pressure is only used for output purposes, since the pressure is not a state variable.)" — Boyce*

`navier_stokes/ex0` is still in the 0.19.0 tree. Its input file defines the exact solution as strings at the top (`U = "1 - 2*(cos(2*PI*(X_0-t))*...)"`, likewise `V` and `P`) and refers to them by name, without quotes, inside the blocks:

```
VelocityInitialConditions {
   nu = MU/RHO
   function_0 = U          // the input parser substitutes the string U; muParser then reads it
   function_1 = V
}
PressureInitialConditions {
   nu = MU/RHO
   function = P            // one function, not function_0: the pressure is a scalar
}
```

Slides 47–48 have the initial-condition blocks of the examples in this series.

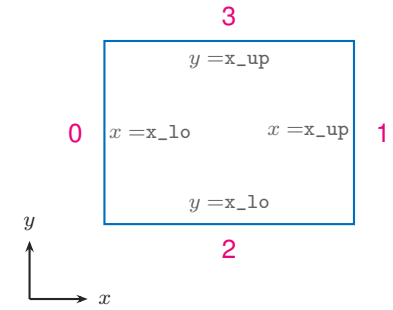
IBAMR BOUNDARY NUMBERING

Look in any `input2d` or `input3d`: each side of the box has a number, used as the suffix of the keys.

- 2D: 0 = x lower (left), 1 = x upper (right), 2 = y lower (bottom), 3 = y upper (top).
- 3D: add 4 = z lower and 5 = z upper.

The rule behind it: side $= 2 \times$ axis $+$ (0 for the lower end, 1 for the upper end), so axis $=$ side$/2$ and lower/upper $=$ side mod 2. The source uses exactly this (`axis = location_index / 2; side = location_index % 2`).

"Lower" and "upper" refer to the coordinate value, not to the picture: in 3D the "bottom" of a box that stands along z is side 4, and in 2D the "floor" is side 2.



axis	lower side	upper side	normal component
x (0)	0	1	u_0: `VelocityBcCoefs_0`
y (1)	2	3	u_1: `VelocityBcCoefs_1`
z (2)	4	5	u_2: `VelocityBcCoefs_2`

THE VELOCITYBCCOEFS BLOCKS: ONE PER COMPONENT, THREE KEYS PER SIDE

There is one block per velocity component: `VelocityBcCoefs_0` for u (x), `_1` for v (y), and in 3D `_2` for w (z). Each block holds, for every side s, the three strings `acoef_function_s`, `bcoef_function_s`, `gcoef_function_s`.

The block on the right (from the 2013–2020 slides) sets the y-velocity to zero on all four sides: no flow through the top and bottom, no slip along the left and right. The x-component block decides whether anything flows at all.

`main.cpp` reads the blocks by name, so the names are fixed: `"VelocityBcCoefs_" << d` for $d = 0, \ldots, \mathrm{NDIM} - 1$, each passed to a `muParserRobinBcCoefs` object (IB/03's `main.cpp`, lines 229–236). What happens when a block is missing: next slide.

```
VelocityBcCoefs_1 {   // y-component of the velocity
   acoef_function_0 = "1.0"
   acoef_function_1 = "1.0"
   acoef_function_2 = "1.0"
   acoef_function_3 = "1.0"

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "0.0"
   bcoef_function_2 = "0.0"
   bcoef_function_3 = "0.0"

   gcoef_function_0 = "0.0"
   gcoef_function_1 = "0.0"
   gcoef_function_2 = "0.0"
   gcoef_function_3 = "0.0"
}
```

HOW THE BLOCKS REACH THE SOLVER, AND WHAT A MISSING BLOCK DOES

```
// IB/03 main.cpp, lines 211-236 (condensed)
const IntVector<NDIM>& periodic_shift = grid_geometry->
  getPeriodicShift();
vector<RobinBcCoefStrategy<NDIM>*> u_bc_coefs(NDIM);
if (periodic_shift.min() > 0)          // fully periodic: no blocks
  needed
{
    for (unsigned int d = 0; d < NDIM; ++d) u_bc_coefs[d] = nullptr;
}
else
{
    for (unsigned int d = 0; d < NDIM; ++d)
    {
        ostringstream bc_coefs_db_name_stream;
        bc_coefs_db_name_stream << "VelocityBcCoefs_" << d;
        u_bc_coefs[d] = new muParserRobinBcCoefs(
            bc_coefs_name, app_initializer->getComponentDatabase(
  bc_coefs_db_name), grid_geometry);
    }
    navier_stokes_integrator->registerPhysicalBoundaryConditions(
  u_bc_coefs);
}
```

- In a fully periodic box (every entry of `periodic_dimension` is 1) the blocks are not read at all; the integrator gets no boundary objects.
- Otherwise one `muParserRobinBcCoefs` is built per component from the block of that name and the set is registered with the fluid integrator; the IB and IBFE `main.cpp` files in this series all contain this loop.
- A missing block is not fatal by itself: `AppInitializer::getComponentDatabase` prints `WARNING: ... Database corresponding to component `VelocityBcCoefs_1' not found in input` and hands over an empty database, so every side then gets the missing-key warning of slide 67 and $a = b = 0$, which the fluid solver rejects at its first use (slide 10). Both warnings are in the log file.

READING A BLOCK: THE x-VELOCITY OF THE TETHERED LEAF (IB/03)

```
VelocityBcCoefs_0 {       // x-component
   U_FLOW = V             // from the top of the file

   acoef_function_0 = "1.0"
   acoef_function_1 = "1.0"
   acoef_function_2 = "1.0"
   acoef_function_3 = "1.0"

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "0.0"
   bcoef_function_2 = "0.0"
   bcoef_function_3 = "0.0"

   gcoef_function_0 = "U_FLOW*tanh(2*t)"
   gcoef_function_1 = "U_FLOW*tanh(2*t)"
   gcoef_function_2 = "U_FLOW*tanh(2*t)"
   gcoef_function_3 = "U_FLOW*tanh(2*t)"
}
```

Line by line, the first half:

- `U_FLOW = V`: a constant for the expressions below, copied from `V = 0.1` at the top of the file (why the copy is needed: slide 25).
- all $a = 1$, $b = 0$: every side prescribes the x-velocity (a velocity condition, slide 10); nothing here is a wall or an open side by itself. What each side does is decided by g, on the next slide.
- the four lines of each key are the four sides of slide 18, in the order left, right, bottom, top.

READING A BLOCK, CONTINUED: WHAT g MAKES OF EACH SIDE

```
   gcoef_function_0 = "U_FLOW*tanh(2*t)"   //
  left:   inflow
   gcoef_function_1 = "U_FLOW*tanh(2*t)"   //
  right:  outflow
   gcoef_function_2 = "U_FLOW*tanh(2*t)"   //
  bottom: sliding wall
   gcoef_function_3 = "U_FLOW*tanh(2*t)"   // top
  :    sliding wall
```

```
VelocityBcCoefs_1 {       // y-component: all
  zero
   acoef_function_0 = "1.0"   ...
   bcoef_function_0 = "0.0"   ...
   gcoef_function_0 = "0.0"   ...
}
```

- $g =$`U_FLOW*tanh(2*t)` on all four sides: the x-velocity rises smoothly from 0 to U (96% of it by $t = 1$ s, slide 28). On side 0 that is inflow, on side 1 outflow, on sides 2 and 3 (where u is tangential) a wall sliding at the stream speed.
- the y-block is all zeros: nothing crosses the top or bottom, and the ends have no y-velocity.
- The course version wrote `0.1*tanh(2*t)` on each line; the constant makes the speed a one-line change at the top of the file.

The result is the uniform stream of slide 31.

WHAT GOES IN THE QUOTES: VARIABLES AND BUILT-IN CONSTANTS

Each string is a muParser expression, evaluated at every boundary point at every time step. `muParserRobinBcCoefs.cpp` defines these names (0.19.0):

Name(s)	Meaning
`t`, `T`	the current time
`X_0`, `X_1`, `X_2` (also `X0`, `x_0`, `x0`)	the coordinates x, y, z of the boundary point being evaluated. On side 0 or 1, `X_0` is constant (it is `x_lo` or `x_up`) and `X_1` varies along the side
`pi`, `Pi`, `PI`	$3.14159\ldots$ (so a `PI` key in the block, as IB/02 has, just redefines the same value)
`X_LOWER_0`, `X_UPPER_0`, ... (also `X_lower0`, `XLower_0`, `xlower0`, ...)	the physical corners of the domain from `CartesianGeometry`, in every spelling the author could think of
any numeric key in the block (`U_FLOW = 0.1`, `f = 5`, `N = 32`)	a constant (slide 25)

The 2013–2020 lectures listed `t`, `X_0`, `X_1`, `X_2`; the rest is from the source. The normal vector is not available as a variable; you know it from the side number.

WHAT GOES IN THE QUOTES: FUNCTIONS AND OPERATORS

muParser 2 (bundled in `ibtk/contrib/muparser`) provides:

- functions: `sin cos tan asin acos atan atan2 sinh cosh tanh asinh acosh atanh exp ln log log2 log10 sqrt abs sign rint`, and the multi-argument `min max sum avg`.
- arithmetic: `+ - * / ^` (power), unary minus, parentheses.
- comparison and logic: `< <= > >= == !=`, `&&`, `||`, `!`; true is 1 and false is 0, so `(t < 1)*t` is a valid piecewise expression.
- the conditional `cond ? a : b` (slide 26).

> Numbers are read as decimals: `1` and `1.0` are the same. Division of two constants is floating-point division (`1/2` is 0.5); the `int()` seen at the top of the input files (`VIZ_INTERVAL = int(END_TIME/(50*DT))`) is SAMRAI's input-file arithmetic, not muParser's, and does not exist inside the quotes.

A name muParser does not know stops the run at start-up with `Unexpected token "<name>" found at position <n>` and the offending expression printed under it (tested on slide 66). The manual, https://beltoforion.de/en/muparser/, has the full list.

CONSTANTS DEFINED IN THE BLOCK, AND THE TRAP AT THE TOP OF THE FILE

Any key in the block whose value is a number becomes a constant the expressions can use (the constructor copies every double and integer key into the parser). That is how the examples parametrize their conditions:

```
VelocityBcCoefs_0 {
   U_FLOW = V                  // IB/03: the stream speed, from V at the top of the file
   PI = 3.141592653589793      // IB/02: also f = 5, d = 1.0, Y_high = 0.5*d, Y_low = -0.5*d
   U0 = U_FLOW                 // IBFE/10: "copied here because the expressions can use only
   V_LID = V                   // IB/08:   constants defined in their block"
   SHEAR = SHEAR_START
   ...
```

> The trap: a name defined at the top level of the input file (`V = 0.1`, `L = 1.0`) is *not* visible inside the quotes. The input file's own arithmetic (`U_FLOW = V`) can see it; muParser, which reads the quoted string later, cannot. Writing `"V*tanh(2*t)"` with `V` only at the top stops the run with `Unexpected token "V" found at position 0` (tested, slide 66). Copy the value into the block, as every example does.

The same rule holds for `VelocityInitialConditions` and the scalar blocks, which use the same parser (`muParserCartGridFunction`). The exception is a top-level *string*: `function_0 = U` without quotes, as in `navier_stokes/ex0` (slide 17), pastes the string in before muParser sees it.

CONDITIONS: THE IF() OF THE OLD LECTURES AND THE ? : OF TODAY

The 2013–2020 slides taught in-line if-statements:

```
if( logical statement, a, b )
```

"if the logical statement is true, do `a`; if false, do `b`", nested as `if(s1, a, if(s2, b, c))`. That was muParser 1's syntax. The bundled muParser 2 has no `if` function; running the course's pulsatile input stops at start-up with

```
muParserRobinBcCoefs::setDataOnPatch():
  error: Unexpected token "if" found at position 0.
  in:    if(t<1, U_FLOW*t, U_FLOW)
```

(An `if` does appear in muParser's source, in its integer-only parser `muParserInt.cpp`; IBAMR does not use that parser.)

Write the condition as a ternary instead:

```
(logical statement) ? a : b
```

Same meaning: if the statement is true the value is `a`, otherwise `b`. Nested:

```
(s1) ? a : ((s2) ? b : c)
```

"if s1 then a, else if s2 then b, else c". Put parentheses around the condition and around a nested ternary; muParser's precedence rules for `?:` are easy to get wrong otherwise. A ramp that reaches U at $t = 1$ and stays there:

```
(t < 1) ? U_FLOW*t : U_FLOW
```

The products-of-logicals form `(t<1)*U_FLOW*t + (t>=1)*U_FLOW` gives the same value and also works.

THE PULSATILE-CHANNEL CONDITION, BEFORE AND AFTER

The course's `4-Example_2DPulsatileChannelFlow` (now IB/02) prescribed, on the left and right ends, a parabolic profile across the channel ($|y| < d/2$), zero outside it, with a peak that pulses in time:

```
// course version (muParser 1 syntax; fails in 0.16 and 0.19 with: Unexpected token "if" found at position 0.)
gcoef_function_0 = "if( X_1 <= -0.5*d,0.0,if( X_1 >= 0.5*d,0.0,4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*( (0.25*d*d) - X_1*X_1 ) ) )"
```

```
// IB/02 as shipped in this series (ternary syntax); the same string on side 1 (gcoef_function_1)
gcoef_function_0 = "(X_1 <= -0.5*d) ? 0.0 : ((X_1 >= 0.5*d) ? 0.0 : 4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*((0.25*d*d) - X_1*X_1))"
```

Reading it from the outside in:

- below the channel ($y \le -d/2$): 0; above it ($y \ge d/2$): 0;
- inside: $\dfrac{4}{d^2}\left(\frac{1}{2} + \frac{1}{2}\sin 2\pi f t\right)\left(\frac{d^2}{4} - y^2\right)$, a parabola with peak 1 at $y = 0$, scaled by a factor that swings between 0 and 1 at $f = 5$ Hz.

The constants `d`, `f` and `PI` are defined in the block (slide 25). Slide 35 draws the profile; slide 34 shows the whole block.

TIME-DEPENDENT CONDITIONS: RAMPS AND OSCILLATIONS

Any expression may use `t`. The patterns in the examples:

- a smooth start, `U*tanh(2*t)`: 0 at $t = 0$, 76% of U at $t = 0.5$, 96% at $t = 1$, 99.9% at $t = 2$. IB/03, IB/08 and IBFE/10 all use it.
- a pulse, `0.5 + 0.5*sin(2*PI*f*t)`: between 0 and 1 with period $1/f$; IB/02. Replace by `sin(2*PI*f*t)` for a flow that reverses.
- a linear ramp that stops, `(t < 1) ? U*t : U`, or a switched-on constant, `(t > t_on) ? U : 0` (slide 26).
- IB/08's lid, `V_LID*(SHEAR + (1 - SHEAR)*tanh(2*t))`: with the block constant `SHEAR = 0` this is the tanh ramp; with `SHEAR = 1` the lid moves at full speed from the first step (used together with a matching initial condition, slide 48).

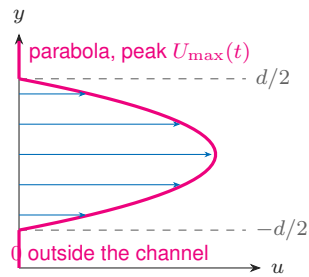
> Why ramp? The fluid starts at rest (slide 47). A boundary velocity that jumps to U at $t = 0$ is an impulsive start: the first steps see an enormous pressure and a velocity field that is not divergence-free near the boundary, the structure gets a hammer blow, and with `error_on_dt_change` the run may stop at once. A $\tanh(2t)$ ramp costs about a second of simulated time and avoids all of that. Chapter 16 (what can go wrong) returns to this.

SPACE-DEPENDENT CONDITIONS: PROFILES ALONG A SIDE

On a side, the coordinate along the side is a variable: `X_1` on sides 0 and 1 (the left and right ends), `X_0` on sides 2 and 3; in 3D two coordinates vary on each face.

- Poiseuille inflow across a channel of width d centred on $y = 0$: $u = U_{\max}\,(1 - 4y^2/d^2)$, which is the IB/02 parabola, $\frac{4}{d^2}(\frac{d^2}{4} - y^2)$, times $U_{\max}$.
- A plug in a window: `(X_1 > Y_low && X_1 < Y_high) ? U : 0`.

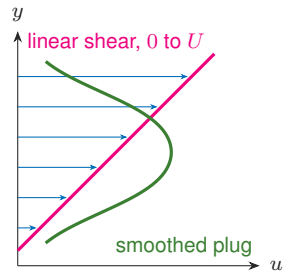
The profile is evaluated at the cell faces of every level that touches the boundary, so it is as sharp as the finest grid there. More profiles on the next slide.



SPACE-DEPENDENT CONDITIONS: MORE PROFILES

- A linear shear: `U*(X_1 - X_LOWER_1)/(X_UPPER_1 - X_LOWER_1)`, using the built-in corner constants of slide 23; 0 at the bottom of the side, U at the top.
- A smoothed window to avoid the sharp corner of the plug: multiply U by `0.5*(1 + tanh((X_1 - Y_low)/w))*0.5*(1 - tanh((X_1 - Y_high)/w))` with w a few cells wide.
- A 3D parabola on side 0 of a square duct: `U*(1 - 4*X_1*X_1/(d*d))*(1 - 4*X_2*X_2/(d*d))`, which is zero on all four duct walls.
- A travelling profile: any function of `X_1 - c*t`; a wave on the side.

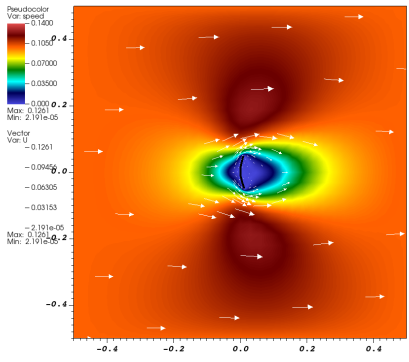
Whatever the profile, the normal velocity on an inflow side should go to zero at the corners of the box where it meets a no-slip wall, or the corner cell has a discontinuous velocity. The parabola does; the plug does not, hence the window.



EXAMPLE: A TETHERED LEAF IN A STREAM (IB/03)

The 2013–2020 “tethered beam in flow” (`4-Example_2DTetheredBeam`): a beam tethered at its centre is placed in a flow that is ramped from 0 to 0.1 by setting *all* boundary conditions to $U(t)$ in x and 0 in y, with U a function of time so that the flow accelerates from rest.

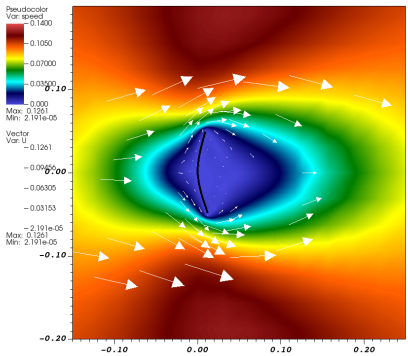
- side 0 (left): inflow at $U \tanh 2t$;
- side 1 (right): outflow at the same speed;
- sides 2 and 3: walls sliding at the stream speed (no boundary layer, so the box can be small);
- $v = 0$ everywhere on the boundary.



Box 1×1 m, $\mu = 4$, $\rho = 1000$, finest grid 512^2.

IB/03 run with IBAMR 0.19.0 (default input, 2 s); VisIt 3.5, `wsl/final/visit_ch7.sh`. Speed $|\mathbf{u}|$ from 0 (blue) to 0.14 m/s (dark red), vectors every 40th cell, the leaf in black.

THE LEAF IN ITS BOX: READING THE PICTURE



The previous picture is speed at $t = 2$ s, with velocity vectors and the leaf in black. The stream is uniform away from the leaf; the fluid is slowed in front of and behind it and speeds up around it. Close up (left): the stagnant region (dark blue) in front and behind, the two fast lobes (red) above and below where the fluid squeezed past the leaf is faster than the stream, and the leaf bent slightly downstream by the flow.

At the box edges the vectors are all the same length and horizontal: that is the boundary condition, $u = U \tanh 4 \approx 0.1$, $v = 0$, on all four sides at this time.

Same run and script as the previous slide, window $-0.15 \le x \le 0.25$, $-0.2 \le y \le 0.2$.

THE LEAF: WHAT THE PRESCRIBED WALLS MEAN, AND FIRST EXPERIMENTS

Because the velocity is prescribed on all four sides, IB/03's box is a wind tunnel whose walls move with the free stream:

- the inflow and outflow fluxes balance exactly (the same U on sides 0 and 1), as slide 12 requires;
- the top and bottom do not grow boundary layers, so the leaf's wake is not confined by them as it would be by no-slip walls;
- but the outlet is *forced* to be uniform: a wake that reaches side 1 is squeezed back to U there. The box must be long enough, or the outlet must be open (slides 13, 42).

> The README of IB/03 suggests the first experiments: make the top and bottom stick (`gcoef_function_2` and `_3` of the x-block to `"0.0"`), or bring the flow in from the top instead (which needs the y-block to carry the stream and the x-block to be zero). Both are one-line edits; both change the wake visibly within the 2 s run.

EXAMPLE: THE PULSATILE CHANNEL (IB/02), THE BLOCK

```
VelocityBcCoefs_0 {
   PI = 3.141592653589793 //  PI-constant
   f = 5 //                   Frequency of pulsation
   d = 1.0 //                 Diameter of Channel
   Y_high = 0.5*d //          Higher y-value of channel [d/2]
   Y_low = -0.5*d //          Lower y-value of channel  [-d/2]

   acoef_function_0 = "1.0"
   acoef_function_1 = "1.0"
   acoef_function_2 = "1.0"
   acoef_function_3 = "1.0"

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "0.0"
   bcoef_function_2 = "0.0"
   bcoef_function_3 = "0.0"

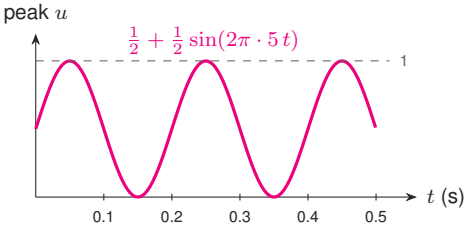
   gcoef_function_0 = "(X_1 <= -0.5*d) ? 0.0 : ((X_1 >= 0.5*d) ? 0.0 :
  4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*((0.25*d*d) - X_1*X_1))"
   gcoef_function_1 = "(X_1 <= -0.5*d) ? 0.0 : ((X_1 >= 0.5*d) ? 0.0 :
  4.0/(d*d)*(0.5 + 0.5*sin(2*PI*f*t))*((0.25*d*d) - X_1*X_1))"
   gcoef_function_2 = "0.0"
   gcoef_function_3 = "0.0"
}
```

This example explores spatially and time-dependent boundary conditions for pulsatile flow through a channel. The channel walls are two rows of target points (chapter 6) inside a 5×5 box; the fluid between them is driven by the profile on the left and right ends.

- the x-block carries the profile on sides 0 and 1 and 0 on the top and bottom of the box;
- the y-block (not shown) is all zeros;
- `Y_high` and `Y_low` are defined but the expression uses `0.5*d` directly; either works.

$\mu = 100$, $\rho = 1000$, finest grid 512^2 over 5 m, so the channel is about 100 cells wide.

THE PULSATILE CHANNEL: WHAT THE EXPRESSION PRESCRIBES



Peak velocity at the centreline versus time: period $1/f = 0.2$ s, maximum 1 at $t = 0.05, 0.25, \ldots$, zero at $t = 0.15, 0.35, \ldots$ The flow never reverses; it pulses between still and full.

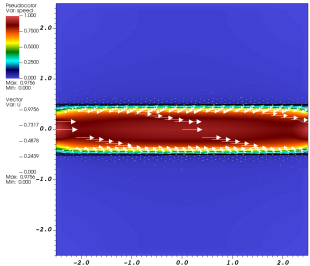
At each instant the profile across the ends is the parabola of slide 29 with that peak; outside $|y| < d/2$ the ends are closed ($u = 0$), so the fluid outside the channel walls is driven only by what leaks around the target-point walls.

What to try (also in IB/02's README):

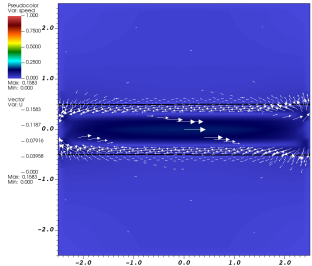
- change `f` and watch the Womersley number $\alpha = \frac{d}{2}\sqrt{2\pi f \rho/\mu}$ change the profile from parabolic to plug-like;
- make the flow reverse with `sin(2*PI*f*t)` instead of `0.5 + 0.5*sin(...)`;
- set `gcoef_function_1` to a traction condition ($a = 0$, $b = 1$, $g = 0$) and let the outlet find its own profile (slides 13, 42).

The next two slides show the channel at a peak and at a trough of the pulse.

THE PULSATILE CHANNEL AT A PEAK AND AT A TROUGH



$t = 0.25$ s, a peak: the inlet factor is 1.

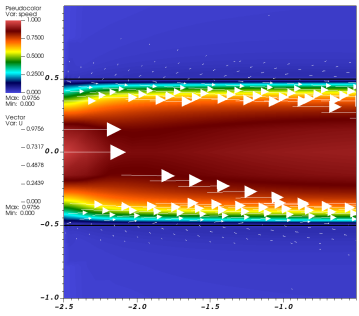


$t = 0.15$ s, a trough: the inlet factor is 0.

Speed from 0 (blue) to 1 (dark red) over the whole 5×5 box; the two channel walls are the black lines at $y = \pm 0.5$. At the peak the channel is full of fast fluid and the box outside the walls is still: the ends are closed there. At the trough the inlet has stopped but the fluid inside has not, and what remains moves fastest *near the walls*, not at the centre.

IB/02 with its shipped input, run to 0.25 s (one and a quarter pulses), IBAMR 0.19.0; VisIt 3.5, `wsl/final/visit_ch7b.sh`.

THE CHANNEL INLET UP CLOSE



The left end of the box at the peak, $-2.5 \le x \le -0.5$. On the boundary ($x = -2.5$) the vectors trace the parabola of the expression: longest at $y = 0$, zero at $y = \pm 0.5$, and zero outside. A short way in, the profile is already flatter in the middle and steeper at the walls.

That is the annular effect of oscillatory flow: with $\nu = \mu/\rho = 0.1$ m^2/s and $f = 5$ Hz the Womersley number is $\alpha = 0.5\sqrt{2\pi \cdot 5/0.1} \approx 8.9$, so the fluid in the core follows the pressure pulse as a plug while a thin Stokes layer at each wall lags it, and at the trough the layers are still moving when the core has stopped (previous slide). The inlet condition forces a parabola, which is the steady profile, not the oscillatory one; the flow adjusts within an entrance length.

A boundary condition prescribes the velocity *on the side*; what the interior does with it is physics, and the first few cells in from the side are where the two meet.

Same run; window $-2.5 \le x \le -0.5$, $-1 \le y \le 1$, $t = 0.25$ s.

EXAMPLE: A GLYCOCALYX FIBRE IN SHEAR (IB/08): PERIODIC SIDES, A FLOOR AND A LID

The 2020 lecture's `4-Example_3DGlycocalyx`: a periodic array of fixed cylinders in shear flow. Periodic conditions in two directions, no slip on the bottom, a prescribed velocity at the top; the cylinder stands in the middle of the domain, coming out of the floor.

- `periodic_dimension = 1, 1, 0`: sides 0–3 are periodic, so their entries in the blocks are ignored (slide 45);
- side 4 (floor, z lower): all components 0;
- side 5 (lid, z upper): $u = V \tanh 2t$, $v = w = 0$.

Only the x-block has a nonzero entry, on side 5. The course version wrote `"0.1*tanh(2*t)"`; the series version uses block constants so that `V` and the option `SHEAR_START` (slide 48) are set once at the top of the file.

```
VelocityBcCoefs_0 {          // x-component
   V_LID = V
   SHEAR = SHEAR_START

   acoef_function_0 = "1.0"    // sides 0-3: periodic,
   acoef_function_1 = "1.0"    // these entries are
   acoef_function_2 = "1.0"    // ignored
   acoef_function_3 = "1.0"
   acoef_function_4 = "1.0"    // floor
   acoef_function_5 = "1.0"    // lid

   bcoef_function_0 = "0.0"
   ...
   bcoef_function_5 = "0.0"

   gcoef_function_0 = "0.0"
   ...
   gcoef_function_4 = "0.0"
   gcoef_function_5 = "V_LID*(SHEAR + (1 - SHEAR)*tanh(2*t))"
}
```

EXAMPLE: THE 3D FLOW TANK (IBFE/10): SIX SIDES, ONE CONSTANT

```
VelocityBcCoefs_0 {          // x-component
   U0 = U_FLOW   // copied here because the expressions
                 // can use only constants defined in their block
   acoef_function_0 = "1.0"
   ...
   acoef_function_5 = "1.0"

   bcoef_function_0 = "0.0"
   ...
   bcoef_function_5 = "0.0"

   gcoef_function_0 = "U0*tanh(2*t)"
   gcoef_function_1 = "U0*tanh(2*t)"
   gcoef_function_2 = "U0*tanh(2*t)"
   gcoef_function_3 = "U0*tanh(2*t)"
   gcoef_function_4 = "U0*tanh(2*t)"
   gcoef_function_5 = "U0*tanh(2*t)"
}
// VelocityBcCoefs_1 and _2: all a = 1, b = 0, g = 0
```

The 3D version of the leaf's conditions: a thin elastic plate in a uniform stream $(U \tanh 2t, 0, 0)$ prescribed on all six faces of a $32 \times 16 \times 16$ tank. The tank is not periodic.

- six sides means six entries per key in 3D; a 2D block has four. Copying a 2D block into a 3D input leaves sides 4 and 5 undefined, which is a warning and $a = b = g = 0$ on those sides (slide 67), not an error.
- the four faces where u is tangential (2–5) are sliding walls again.
- the domain is longer in x and the plate sits a quarter of the way in; the geometry block that does this is on slide 55.

The IBFE method does not change the Eulerian boundary conditions at all: the blocks are read by the same `muParserRobinBcCoefs` and registered with the same Navier–Stokes integrator as in the IB examples.

SCALAR CONDITIONS: THE TEMPERATURE OF IBFE/12

```
TemperatureBcCoefs {
   L = L

   acoef_function_0 = "1.0"
   acoef_function_1 = "1.0"
   acoef_function_2 = "1.0"
   acoef_function_3 = "1.0"

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "0.0"
   bcoef_function_2 = "0.0"
   bcoef_function_3 = "0.0"

   gcoef_function_0 = "..."   // the initial T, so that the
   gcoef_function_1 = "..."   // boundary agrees with the
   gcoef_function_2 = "..."   // interior at t = 0
   gcoef_function_3 = "..."
}
```

```
// main.cpp of IBFE/12
T_bc_coef = std::make_unique<muParserRobinBcCoefs>("T_bc_coef",
    app_initializer->getComponentDatabase("TemperatureBcCoefs"),
  grid_geometry);
```

An advected and diffused scalar (temperature, concentration) has a block of the same shape, under whatever name `main.cpp` asks for. Two differences from the velocity:

- the coefficients are passed straight to the diffusion solver, which handles a general Robin condition, so a and b may *both* be nonzero: $a\,T + b\,\partial T/\partial n = g$ with, say, $a = h$, $b = k$ is a convective (Newton cooling) condition. The upstream `adv_diff` examples use Neumann ($b = 1$) sides;
- there is no “traction” meaning; $b = 1$, $g = 0$ is simply an insulated side, and $a = 1$ a prescribed value.

IBFE/12’s domain is periodic in both directions, so its block is read but not used; it is there so that `periodic_dimension = 0, 0` works without further edits. Chapter 15 covers the advection–diffusion solver itself.

COMPATIBILITY: NET FLUX, THE PRESSURE LEVEL AND NORMALIZE_PRESSURE

Two consistency rules follow from incompressibility and from the pressure having no condition of its own.

1. Net flux. If every side is a velocity condition or periodic, $\nabla \cdot \mathbf{u} = 0$ integrated over the box forces $\oint u_n \, dS = 0$. The solver cannot satisfy an inflow with closed walls; the projection will produce something, but not a solution of your problem. An open side (traction) removes the constraint: the flux through it is whatever balances the rest.
2. The pressure level. With velocity conditions or periodic sides everywhere, the pressure is defined only up to a constant. `normalize_pressure = TRUE` pins it by subtracting the mean every step; `FALSE` (the default) lets the solver's own null-space handling decide, and the level can drift without affecting the velocity. The IB examples in this series set it `TRUE`, the IBFE examples `FALSE`; both run. With at least one traction side the boundary fixes the level (slide 14), and the mean must *not* be removed: `normalize_pressure = FALSE`, as in the upstream open-boundary examples (`IB/explicit/ex2`'s stabilized cylinder, `IIM/ex9`).

> A related key, `normalize_velocity` (default `FALSE`), removes the mean velocity in fully periodic domains where it is otherwise undetermined (a periodic box with no walls can drift). Both are keys of the fluid integrator's block; the examples pass `NORMALIZE_PRESSURE` from the top of the file.

OPEN BOUNDARIES IN PRACTICE: THE UPSTREAM CYLINDER (IB/EXPLICIT/EX2)

```
NORMALIZE_PRESSURE  = FALSE   // an open side fixes the level

VelocityBcCoefs_0 {           // x-component
   t_half = 0.5
   tau = 0.125

   acoef_function_0 = "1.0"   // left: velocity (inflow)
   acoef_function_1 = "0.0"   // right: traction (open)
   acoef_function_2 = "0.0"   // bottom: traction
   acoef_function_3 = "0.0"   // top: traction

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "1.0"
   bcoef_function_2 = "1.0"
   bcoef_function_3 = "1.0"

   gcoef_function_0 = "(tanh((t-t_half)/tau)+tanh(t_half/tau))/(1+
  tanh(t_half/tau))"
   gcoef_function_1 = "0.0"
   gcoef_function_2 = "0.0"
   gcoef_function_3 = "0.0"
}
```

`examples/IB/explicit/ex2/input2d.cylinder_stabilized` in the 0.19.0 tree: flow past a cylinder with an inflow on the left and an open right side. The x-block:

- side 0: $u =$ a smooth ramp from 0 to 1 centred at $t = 0.5$ with width 0.125 (another way to write the start-up of slide 28);
- side 1: $(a, b) = (0, 1)$, $g = 0$: zero normal stress, so $p \approx 0$ there (slide 14); the outlet velocity is whatever the interior produces;
- sides 2 and 3: $(a, b) = (0, 1)$, $g = 0$ on u, which is *tangential* there: zero shear stress along the top and bottom.

The y-block on the next slide completes the picture.

THE UPSTREAM CYLINDER: FREE-SLIP WALLS AND A NUDGE TO START SHEDDING

```
VelocityBcCoefs_1 {              // y-component
   acoef_function_0 = "1.0"      // left: velocity
   acoef_function_1 = "0.0"      // right: traction
   acoef_function_2 = "1.0"      // bottom: velocity
   acoef_function_3 = "1.0"      // top: velocity

   bcoef_function_0 = "0.0"
   bcoef_function_1 = "1.0"
   bcoef_function_2 = "0.0"
   bcoef_function_3 = "0.0"

   gcoef_function_0 = "cos(pi*X_1/16.0)*exp(-0.5*t)"
   gcoef_function_1 = "0.0"
   gcoef_function_2 = "0.0"
   gcoef_function_3 = "0.0"
}
```

- side 0: $v = \cos(\pi y/16)\, e^{-t/2}$, a small transverse velocity that dies away in a few seconds: a nudge that breaks the symmetry so that vortex shedding starts without waiting for round-off to do it;
- side 1: traction with $g = 0$ on v too, so the outlet is open in *both* components: zero normal stress and zero shear, the fully open outlet of slide 13;
- sides 2 and 3: $v = 0$, a velocity condition on the *normal* component (no flow through), combined with the zero shear on u from the previous slide: free-slip walls, which confine the flow without boundary layers.

The wake leaves through the right side at whatever velocity it has; nothing squeezes it back to uniform. The price is possible backflow at the outlet, which the next slide handles. `NORMALIZE_PRESSURE = FALSE` because side 1 fixes the pressure level (slide 41).

BACKFLOW AT AN OPEN SIDE: THE OPEN-BOUNDARY STABILIZER

An open outlet lets fluid come *in* wherever the wake's velocity points inward, and an open inlet lets it leave. Either can feed an instability. IBAMR's remedy is `StaggeredStokesOpenBoundaryStabilizer`, a body force applied in a layer next to the side that pushes the offending velocity back toward zero. From the source:

```
if ((d_inflow_bdry[location_index] && U * n > 0.0) || (d_outflow_bdry[location_index] && U * n < 0.0))
    (*F_data)(i_s) = smooth_kernel((x - x_bdry) / d_width[location_index]) * kappa * (0.0 - U);
```

In the input file (`IB/explicit/ex2`'s stabilized cylinder; `IIM/ex9` does the same for sides 4 and 5 with `"INFLOW"` on 4):

```
BoundaryStabilization {
   stabilization_type_1 = "OUTFLOW"
   width_1 = 4.0*DX_FINEST
}
```

Keys `stabilization_type_s` (`"INFLOW"`, `"OUTFLOW"` or `"NONE"`) and `width_s` per side; four fine cells is the usual width.

It is registered in `main.cpp`, not automatically; `IB/explicit/ex2` does so only when the block exists and the solver is staggered:

```
if (solver_type == "STAGGERED" && input_db->keyExists("
  BoundaryStabilization"))
    time_integrator->registerBodyForceFunction(
        new StaggeredStokesOpenBoundaryStabilizer(
            "BoundaryStabilization",
            app_initializer->getComponentDatabase("
  BoundaryStabilization"),
            navier_stokes_integrator, grid_geometry));
```

The examples in this series do not register it (they have no open sides). Chapter 15 reads `IB/explicit/ex2` in full.

PERIODIC SIDES: PERIODIC_DIMENSION OVERRIDES EVERYTHING

```
CartesianGeometry {
   domain_boxes = [ (0,0) , (N - 1,N - 1) ]
   x_lo = -0.5*L, -0.5*L   // lower end of computational domain.
   x_up =  0.5*L,  0.5*L   // upper end of computational domain.
   periodic_dimension = 1, 1
}
```

`periodic_dimension` turns on periodic boundary conditions, one flag per direction: `1` is on, `0` is off. A periodic direction identifies its two sides: what leaves through side 1 enters through side 0. The block above (the 2013–2020 slide) makes the whole box periodic; IB/08 uses `1, 1, 0`; IBFE/12 uses `1, 1`; the other examples in this series use all zeros and the `VelocityBcCoefs` blocks.

What it means in practice:

- on a periodic direction's two sides the `VelocityBcCoefs` entries are ignored, silently. Tested: IB/03 with `periodic_dimension = 1, 0` and the inflow still written on sides 0 and 1 runs without a warning, and no inflow happens (slide 67);
- the Lagrangian points cross a periodic side and re-enter on the other (the `LDataManager` applies the periodic shift); a structure may straddle it.

More consequences on the next slide.

PERIODIC SIDES: CONSEQUENCES

- In a fully periodic box there are no walls to hold the fluid still, so the mean velocity can drift (`normalize_velocity`, slide 41), and nothing can be “in flow” unless a body force or a moving structure drives it. IBFE/12 (periodic both ways) moves its scalar with a prescribed velocity field, not with a boundary condition.
- The periodic length is `x_up - x_lo`, which must be larger than the structure plus its wake if the images are not to interact. IB/08 *wants* the interaction: the fibre is one of an infinite array, $1/8$ m apart.
- A direction can be periodic while the others carry boundary conditions: IB/08 is periodic in x and y and has a floor and a lid in z. The entries for sides 0–3 must still be present in the blocks (or the missing-key warning appears), and they may say anything.
- Periodic in the *flow* direction turns an inflow–outflow problem into a driven one: the leaf with `periodic_dimension = 1, 0` sees its own wake arrive from upstream, and the only thing moving the fluid is the sliding top and bottom (the tested case of slide 67).
- Refinement patches may cross a periodic side; the gridding algorithm handles the wrap, and VisIt draws the levels in the box as usual.

INITIAL CONDITIONS

```
VelocityInitialConditions {        // IB/03: at rest
   function_0 = "0.0"
   function_1 = "0.0"
}

VelocityInitialConditions {        // IB/08: an option
   SHEAR = SHEAR_START
   V0 = V
   L0 = L
   function_0 = "SHEAR*V0*(X_2 + 0.5*L0)/L0"
   function_1 = "0.0"
   function_2 = "0.0"
}
```

`main.cpp` registers the block if it exists (`keyExists("VelocityInitialConditions")`), likewise `PressureInitialConditions` with its single `function`; the examples in this series have only the velocity block.

`function_d` is the d-th velocity component at $t = 0$ as an expression in `X_0`, `X_1`, `X_2` (and `t`, which is 0), with the same parser and the same block-constant rule as the boundary blocks.

- Start from rest unless you have a reason not to: it is consistent with any ramped boundary condition, and chapter 16's first rule for an unstable run is "start from zero velocity".
- The initial velocity should agree with the boundary condition at $t = 0$; a mismatch is a (one-step) impulsive start.

INITIAL CONDITIONS, CONTINUED: A NON-TRIVIAL START

- IB/08's option: with `SHEAR_START = 1` the fluid starts in the linear shear profile that the lid would produce, $u = V(z + L/2)/L$, and the lid moves at V from the first step (slide 28); the fibre feels the flow at once instead of after the viscous start-up, which takes 0.2 s or more to reach the floor (the README has the 1D solution and the measured profiles).
- The two must match: the initial profile has $u = V$ at the lid and 0 at the floor, which is exactly what the boundary blocks say at $t = 0$ with `SHEAR = 1`. Change one and not the other and the first step is impulsive.
- An initial condition that does not satisfy $\nabla \cdot \mathbf{u} = 0$ is projected at the first step; what you see in frame 0 is what you wrote, what you see in frame 1 is the divergence-free part of it.
- The pressure initial condition is used only for the first output frame (slide 17); `navier_stokes/ex0` sets it because the exact solution is known and the error is measured from frame 0.
- A structure's initial state is the `.vertex` file (chapter 6), not an expression; the fluid and the structure start independently, and a structure that starts stretched or bent releases its energy into fluid that is at rest.

Part II: the domain and its grid

The `CartesianGeometry` block; levels, refinement ratio and cell counts; two worked examples (a tall thin box, a long tank); the divisibility rules; patch sizes; tagging and regridding; keeping the structure off the edge; a first guess at Δt; output intervals and disk space; checking before a long run.

This is the 2017 “Using different domain dimensions” deck and Part II of the 2020 Lecture 4, with the 2015 lecture’s cautions on grid refinement.

CARTESIANGEOMETRY: THE INDEX BOX AND THE PHYSICAL BOX

```
CartesianGeometry {
   domain_boxes = [ (0,0,0) , (N/8 - 1,N/8 - 1,N - 1) ]
   x_lo = -L/16, -L/16, -L/2   // lower end of computational
  domain.
   x_up =  L/16,  L/16,  L/2   // upper end of computational
  domain.
   periodic_dimension = 1, 1, 0
}
```

(IB/08; a 2D block has pairs instead of triples.)

- `domain_boxes`: the cells of the coarsest level, as an index box from $(0, 0, 0)$ to $(M_x - 1, M_y - 1, M_z - 1)$ where M_d is the number of coarsest cells in direction d. A list, because SAMRAI allows several boxes; IBAMR uses one.
- `x_lo`, `x_up`: the physical corners, in metres (or whatever unit the rest of the file uses). The lower corner sets $x_{\min}, y_{\min}, (z_{\min})$ and the upper one the maxima.
- `periodic_dimension`: slide 45.

The cell size on the coarsest level is $(x_{\text{up}} - x_{\text{lo}})_d / M_d$ in each direction, and on level ℓ it is that divided by the refinement ratio ℓ times. Nothing forces the three cell sizes to be equal, but the IB kernels and every example assume square (cubic) cells: keep the physical lengths in the same proportion as the cell counts. IB/08: $\frac{1}{8} : \frac{1}{8} : 1$ and $4 : 4 : 32$. IBFE/10: $32 : 16 : 16$ and $32 : 16 : 16$.

The coarsest box is what you type; the finer levels are made by the gridding algorithm (next slide) and never typed. So “changing the size of the domain” means changing `x_lo`/`x_up` *and* `domain_boxes` together, or the cells change shape.

LEVELS, REFINEMENT RATIO, N AND NFINEST

```
MAX_LEVELS = 3                              // maximum number of
  levels
REF_RATIO  = 4                              // refinement ratio
  between levels
N = 32                                      // cells on the
  coarsest level
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N    // cells on the finest
  level
DX = (1.0*L)/NFINEST                        // finest mesh width (m)


GriddingAlgorithm {
   max_levels = MAX_LEVELS
   ratio_to_coarser {
      level_1 = REF_RATIO,REF_RATIO
      level_2 = REF_RATIO,REF_RATIO
      ...
   }
   ...
}
```

IBAMR refines the grid near the structure (and, optionally, where the vorticity is large). `MAX_LEVELS` is the number of levels including the coarsest; `REF_RATIO` the ratio of cell counts between a level and the next coarser one, the same in every direction.

$$\mathrm{NFINEST} = \mathrm{REF_RATIO}^{\mathrm{MAX_LEVELS}-1} \times N.$$

`DX` is the finest mesh width; it is what the structure's point spacing (chapter 6, $\Delta s \approx \Delta x/2$) and the Δt guess (slide 62) refer to. The `ratio_to_coarser` list must have an entry for every level above the coarsest; extra entries (the UNC inputs list five) are harmless.

LEVELS AND RATIOS: THE 2015 EXAMPLE AND WHAT THE EXAMPLES USE

The 2015 lecture's example: a 1×1 domain with $N = 8$, four levels and ratio 4 gives $8 \to 32 \to 128 \to 512$: the finest grid would be 512×512 *if it covered the whole box*; it covers only the tagged region. "Setting `MAX_LEVELS` to 4 is a good default; `REF_RATIO = 4` is a rather strict refinement ratio."

example	`N`	levels	finest	note
IB/01, 06, 07	8	4	512	the 2015 recipe
IB/02, 03	32	3	512	same finest grid, one level fewer
IB/04 (moving cylinders)	16	4	1024	the finest grid in the series
IB/08 (3D)	32	3	$64 \times 64 \times 512$	`N` counts the long direction
IBFE/01, 05, 06, 08, 12	32	2	128	two levels: the finite-element mesh carries the resolution
IBFE/10 (3D)	16	3	$512 \times 256 \times 256$	`N` counts the short directions

A ratio of 2 with more levels gives a gentler nesting and more regrid work; a ratio of 4 with fewer levels is what every example here uses. The number of levels is the real cost lever: each level is another full set of patches, ghost cells and solver sweeps.

WORKED EXAMPLE 1: A TALL THIN BOX FOR THE GLYCOCALYX (2017, 2020)

"Making the lengths of the domain different sizes." The changes are made in the input file, and you have to think the refinement levels through so that they make sense. The target: a box of physical size $\frac{1}{8} \times \frac{1}{8} \times 1$ with a finest grid of $64 \times 64 \times 512$.

1. Let `NFINEST` be the number of finest cells in the longest direction: 512. In the two short directions there are $\mathrm{NFINEST}/8 = 64$.
2. Three levels, because the grid is so small in two directions; ratio 4. Then the coarsest level in the long direction has $N = 512 \times (\frac{1}{4})^{3-1} = 32$ cells, and in the short directions $N/8 = 4$.
3. The cells are cubes: $1/512$ in every direction, because $\frac{1}{8}/64 = 1/512$ too.
4. The index box runs from 0 to $M - 1$ in each direction, M being the coarsest count: from $(0, 0, 0)$ to $(3, 3, 31)$, written with the constants as on the next slide.
5. The physical box is centred: from $(-\frac{1}{16}, -\frac{1}{16}, -\frac{1}{2})$ to $(\frac{1}{16}, \frac{1}{16}, \frac{1}{2})$, with $L = 1$.

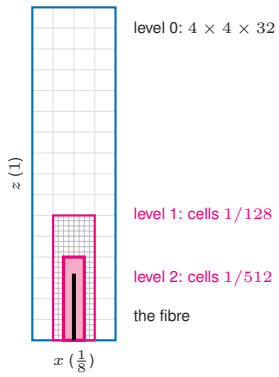
> Four cells across the coarsest level is as low as one should go (slide 56). The 2020 summary: "make sure you don't have a coarsest level of size 1 (or even 2) in any dimension."

WORKED EXAMPLE 1: THE INPUT LINES, AND THE LEVELS IN A PICTURE

```
MAX_LEVELS = 3        // three levels: the grid is small in two
  directions
REF_RATIO  = 4
N = 32                // coarsest cells in the LONGEST direction
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N   // 512 in the longest
  direction

CartesianGeometry {
   domain_boxes = [ (0,0,0) , (N/8 - 1,N/8 - 1,N - 1) ]
   x_lo = -L/16, -L/16, -L/2
   x_up =  L/16,  L/16,  L/2
   periodic_dimension = 1, 1, 0
}
```

In IB/08 as shipped, with its fibre of 13,600 target points standing on the floor; the finest level covers the fibre and a margin, the coarsest the whole column.



A y-slice: the coarsest level fills the column, the finer levels nest around the fibre (schematic; the real patches are chosen by the gridding algorithm, slide 57).

WORKED EXAMPLE 2: A LONG TANK WITH THE PLATE A QUARTER OF THE WAY IN (IBFE/10)

```
L = 16.0                // width of computational domain
MAX_LEVELS = 3
REF_RATIO  = 4
N = 16                  // coarsest cells across (y and z)
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N   // 256 across
DX  = L/NFINEST         // 1/16

CartesianGeometry {
   domain_boxes = [ (0,0,0),(2*N - 1,N - 1,N - 1) ]
   x_lo = -0.5*L,-0.5*L,-0.5*L
   x_up =  1.5*L, 0.5*L, 0.5*L
   periodic_dimension = 0,0,0
}
```

Here the short directions carry `N`, and the long one is `2*N`: a $32 \times 16 \times 16$ coarsest grid for a $32 \times 16 \times 16$ tank, cubes of side 1 on level 0 and $1/16$ on level 2.

- `x_lo` and `x_up` need not be symmetric: x runs from -8 to 24, so the plate at the origin sits 8 units from the inlet and 24 from the outlet, where its wake has room.
- `NFINEST` is a name, not a rule: it counts the finest cells *across* here and *along* in IB/08. What matters is that `DX` is the same in every direction.
- the three cell counts $32, 16, 16$ are all multiples of $\mathrm{REF_RATIO}^2 = 16$, so each level's cells line up with the next (slide 56).

THE RULES: DIVISIBILITY, MINIMUM SIZES, SQUARE CELLS

From the 2020 summary slide, with the reasons:

1. Each coarsest cell count must be a whole number, and the finest count in each direction must be divisible by $\mathrm{REF_RATIO}^{\mathrm{MAX_LEVELS}-1}$: the levels are built by refining whole coarse cells, so the finest grid is the coarsest times a power of the ratio, in every direction. Decide the finest resolution you want, divide down, and round the coarsest count *up* to a whole number.
2. No coarsest level of size 1, or even 2, in any direction. The coarsest level must hold the smallest patch (slide 57) and SAMRAI's ghost-cell layers; the multigrid solvers want something to coarsen; and the refined region cannot be placed with any accuracy when the coarse cells are as big as the structure. Four is a practical minimum (IB/08); eight is comfortable.
3. Keep the cells square: physical lengths in the same ratio as cell counts (slide 50).
4. The patch sizes must fit: the smallest patch must divide the coarsest box in each direction, and the largest patch must be at least the smallest (slide 57).

> A quick check for a new domain: write M_x, M_y, M_z (coarsest counts) and $\mathrm{NFINEST}_d = M_d\, r^{\ell-1}$; confirm $(x_{\mathrm{up}} - x_{\mathrm{lo}})_d / \mathrm{NFINEST}_d$ is the same number for every d; confirm each $M_d \geq 4$ and divisible by the smallest patch size.

PATCH SIZES: LARGEST_PATCH_SIZE AND SMALLEST_PATCH_SIZE

```
largest_patch_size {
   level_0 = NFINEST/8, NFINEST/8, NFINEST/8
   // NOTE: all finer levels will use same values as level_0...

}
smallest_patch_size {
   level_0 = 4, 4, 4
   // NOTE: all finer levels will use same values as level_0...

}
```

(IB/08.) The 2020 heuristic: “the largest patch size should be a cube of the finest grid that fills the domain in the shortest direction(s); the smallest patch size should be a cube of the coarsest grid that fills the domain in the shortest directions”: 64^3 and 4^3 here.

A level is made of rectangular patches; these keys bound their size in cells. The limits are per level but, with only `level_0` given, apply to all levels.

- smaller patches: better load balance across processes, more boundary and ghost-cell overhead;
- larger patches: fewer, bigger blocks; fine for one process, poor for sixteen.

The examples disagree, which shows these are heuristics, not rules:

example	largest	smallest
IB/01, 02, 04, 06, 07	`NFINEST, NFINEST`	`8, 8`
IB/03	`NFINEST, NFINEST`	`32, 32`
IB/08	`NFINEST/8` cubed	`4, 4, 4`
IBFE/01, 05, 06, 08, 12	`512, 512`	`8, 8`
IBFE/10	`256, 256, 256`	`8, 8, 8`

The smallest patch must divide the coarsest box: 32×32 on IB/03’s 32^2 coarsest grid is one patch; 4^3 on $4 \times 4 \times 32$ gives eight. A largest patch bigger than the grid (the IBFE 512^2 on a 128^2 finest grid) is simply no limit.

THE REST OF THE GRIDDING BLOCK: EFFICIENCY AND THE TAGGING METHOD

```
GriddingAlgorithm {
   ...
   efficiency_tolerance = 0.85e0  // min % of tag cells in new
  patch level
   combine_efficiency   = 0.85e0  // chop box if sum of volumes
  of smaller
                                  // boxes < efficiency * vol of
  large box
}

StandardTagAndInitialize {
   tagging_method = "GRADIENT_DETECTOR"
}
```

- `efficiency_tolerance`: a new patch must be at least this fraction tagged cells (0.85 in the IB examples, 0.80 in IBFE/10); lower values give fewer, fatter patches with more untagged cells refined.
- `combine_efficiency`: when to split a box into smaller ones rather than keep one with untagged corners.
- `tagging_method = "GRADIENT_DETECTOR"`: the integrator tags cells itself; for IB that means the cells that contain Lagrangian points (plus the vorticity test on slide 60, if on). The name is SAMRAI's; IBAMR's integrators implement the "gradient detector" call-back with their own criteria.

THE REST OF THE GRIDDING BLOCK: THE TAG BUFFER AND WHEN TO REGRID

```
IBHierarchyIntegrator {
   ...
   regrid_cfl_interval = REGRID_CFL_INTERVAL   // 0.5 in the
  examples
   tag_buffer          = TAG_BUFFER            // 1 in the
  examples
}
```

```
IBHierarchyIntegrator::initializePatchHierarchy(): tag_buffer = 1
   3
```

(the log line that confirms the setting; the second number is the buffer for the next level, computed from the first)

- `tag_buffer`: how many extra cells around a tagged cell are refined with it, per level. The buffer is what lets the structure move between regrids without leaving the fine level.
- `regrid_cfl_interval`: regrid whenever any Lagrangian point could have moved this many mesh widths since the last regrid (the IB integrators); the alternative key `regrid_interval` regrids every so many steps. Regridding is expensive (new patches, data transfer, solver set-up), so 0.5 with a buffer of 1 is the usual balance.
- A structure that moves fast needs a larger buffer or a smaller interval; the symptom of too little is a point that steps off the finest level, which the log reports and the run may not survive (chapter 16).

REFINING ON VORTICITY (THE 2015 CAUTIONS)

```
using_vorticity_tagging = TRUE
vorticity_abs_thresh = 0.25
// or, instead of the absolute threshold:
vorticity_rel_thresh = 0.25
```

(Keys of the fluid integrator's block, `INSStaggeredHierarchyIntegrator`; the examples set `VORTICITY_TAGGING = FALSE` at the top of the file and pass it in.)

"You can also set vortices for local refinement. If you set this too small, you will tag the whole domain for refinement! If it is too big, you won't resolve the vortex wake."

With vorticity tagging on, a cell is also refined when $|\omega|$ exceeds the threshold: absolute, in the vorticity's units, or relative to the maximum vorticity on the level (`vorticity_rel_thresh`; both keys may be given per level as a list).

- the wake of a flapping or shedding body is then followed by the fine grid instead of being resolved only where the structure is;
- too small a threshold refines everything (the run slows to the cost of a uniform fine grid and the levels stop meaning anything); too large a threshold refines nothing beyond the structure;
- the right value depends on U/L: start with a relative threshold of a few tenths and look at the levels in VisIt (chapter 5's level plot).

The examples in this series leave it off; the leaf's wake is short and the boxes are small.

KEEP THE STRUCTURE OFF THE EDGE OF THE BOX

"You cannot have a boundary right on the edge of the domain initially. Move it off the edge of the domain by ds." The 2020 generator for the glycocalyx fibre (reformatted):

```
offset = -0.5*L + dq;   % fibre base, dq above the floor (m)
for q = 0:npts_height-1
  for s = 0:npts_circum-1
    X(1) = glyco_radius*cos(s*2*pi/npts_circum);
    X(2) = glyco_radius*sin(s*2*pi/npts_circum);
    X(3) = offset + q*dq;
    fprintf(vertex_fid, '%1.16e %1.16e %1.16e\n', X(1), X(2), X(3)
  );
  end
end
```

The first ring sits at $z = -L/2 + dq$, one point spacing above the floor, not on it.

Why:

- the IB kernel (chapter 5) spreads each point's force over 4 cells (± 2 around it); for a point *on* the boundary half of that support is outside the box, in ghost cells, and the force is partly lost or reflected in a way that depends on the boundary condition;
- a point exactly on `x_lo` can be assigned to the wrong side of a periodic boundary or to no patch at all after rounding;
- a point outside the box is a fatal error at start-up or at the first regrid.

The same goes for the top: IB/08's fibre stops well below the lid. And for a structure that will *move*: leave room for the whole motion, plus the two-cell kernel margin, on every side (IB/04's cylinders and IB/07's plates are placed with their paths in mind).

A FIRST GUESS AT Δt (AND WHAT THE EXAMPLES USE)

The 2015 lecture: "we can also use this to determine `dx` and take a guess at `dt`. If each flap or pulse is one unit in time, a good guess for `dt` is 10^{-5}."

```
N = 8                                          // coarsest grid
   spacing
NFINEST = (REF_RATIO^(MAX_LEVELS - 1))*N  // finest   grid
   spacing
DX = (1.0*L)/NFINEST                           // Cartesian mesh width
    (m)
DT = 3.125e-2/NFINEST                          // timestep duration (s)
```

That rule gives 6.1×10^{-5} at $\mathrm{NFINEST} = 512$ and 1.2×10^{-4} at 256: a CFL-type bound, $\Delta t \propto \Delta x$. The examples set `DT` by hand instead (right).

For IB/03, $U\,\Delta t/\Delta x = 0.1 \times 10^{-4} \times 512 = 0.005$: the step is 60 times below the CFL limit (`CFL_MAX = 0.3`). The step is set by the stiffness of the springs and targets, not by the flow (chapter 5; the measured limits are in the READMEs).

example	finest	`DX`	`DT`
IB/01, 06, 07	512	$L/512$	10^{-4}
IB/02, 03	512	$L/512$	10^{-4}
IB/04 (moving cylinders)	1024	$L/1024$	2×10^{-5}
IB/08 (3D)	512	$1/512$	10^{-5}
IBFE/01, 06, 12	128	$L/128$	10^{-3}
IBFE/05, 08	128	$L/128$	5×10^{-4}
IBFE/10 (3D)	256	$1/16$	10^{-4}

So: start from the nearest example's `DT`, halve it if the structure explodes, and let `CFL_MAX` catch the rest. `DT` is passed to the integrator as `dt_max`; the integrator may take smaller steps, never larger ones.

OUTPUTS AND THE HARD DRIVE

```
// visualization dump parameters
   viz_writer            = "VisIt"
   viz_dump_interval     = VIZ_INTERVAL   // steps; 0 turns it
  off
   viz_dump_dirname      = "viz_IB2d"
   visit_number_procs_per_file = 1

// hierarchy data dump parameters
   data_dump_interval    = DATA_INTERVAL  // steps; 0 turns it
  off
   data_dump_dirname     = "hier_data_IB2d"
```

```
// at the top of the examples' input files
VIZ_INTERVAL  = int(END_TIME/(50*DT))   // 50 frames per run
DATA_INTERVAL = int(END_TIME/(50*DT))
```

"Watch this so that you don't fill up the hard drive!" (2015). Both intervals are in time steps, not in time: with `DT = 1e-4` and `viz_dump_interval = 10000`, as the course inputs had, a 2 s run writes two frames; with `END_TIME = 20` it writes twenty.

- the examples in this series compute the intervals from the run length: fifty frames per run whatever `DT` is;
- `hier_data` files hold the positions and forces of every point (chapter 11) and grow with the structure; fifty of them is plenty.

END TIME, STEP COUNT, AND FRAME SIZES

```
IBHierarchyIntegrator {
   start_time            = 0.0e0
   end_time              = END_TIME          // 2.0 in IB/03
   grow_dt               = 2.0e0
   max_integrator_steps = 10000000
   ...
}
```

“This is another way to fill up the hard drive, so be careful! `end_time` sets the final time in seconds; `max_integrator_steps` sets the maximum number of time steps.” (2015)

- `end_time` and `max_integrator_steps`: the run stops at whichever comes first. The course’s `end_time = 20` with `DT = 1e-4` was 200,000 steps, hours on a laptop; the series uses 2 s.
- `grow_dt`: how fast the step may grow back after the integrator has shrunk it; with `error_on_dt_change` it never gets the chance (chapter 5).
- A 512^2 VisIt frame with $\mathbf{u}$, p, ω and $\nabla \cdot \mathbf{u}$ is a few megabytes with the AMR hierarchy, more if the fine level covers much of the box; a $64 \times 64 \times 512$ 3D frame is tens of megabytes. Count before you set the interval: frames $\times$ size, plus the restart files if those are on.

CHECKING THE SET-UP BEFORE A LONG RUN

Ten minutes of checking saves an overnight run that was never going to work.

1. Read the blocks aloud: for each component and side, say "on the left, u is prescribed as ...". Every side appears in every block; the number of sides is $2\,\mathrm{NDIM}$.
2. Run two steps (set `END_TIME` to twice `DT`) on one process and read the log file (`IB2d.log`, `IB3d.log`): the warnings about missing keys are there and only there (slide 67); the `tag_buffer` line and the level set-up lines show that the hierarchy was built; the structure files were read (chapter 6).
3. Open the first frame in VisIt: plot `U` as vectors or its magnitude, and the levels (chapter 5's `levels` subset). The boundary velocity should be what you wrote; the fine level should sit on the structure with a margin; the structure should be inside the box.
4. Check the numbers: $(x_{\mathrm{up}} - x_{\mathrm{lo}})/\mathrm{NFINEST}$ the same in every direction; `DX` against the structure's spacing; net flux zero if all sides are prescribed; `normalize_pressure` matching the kind of sides you have.
5. Then run for a short physical time (one ramp, one pulse) and look again before the long run.

WHAT CAN GO WRONG: STOPS AT START-UP (TESTED)

Each case is IB/03 with one edit, IBAMR 0.19.0, one process (`wsl/final/ch7_errors.sh`). "Stops" means an abort before the first step.

The edit	What IBAMR says
a top-level name inside the quotes: `"V*tanh(2*t)"` with `V` defined only at the top of the file	stops: `muParserRobinBcCoefs::setDataOnPatch(): error: Unexpected token "V" found at position 0. in: V*tanh(2*t)` (slide 25)
the course's `if()` syntax: `"if(t<1, U_FLOW*t, U_FLOW)"`	stops: `Unexpected token "if" found at position 0.` (slide 26)
$a = b = 0.5$ on one side	stops: `Program abort called in file INSProjectionBcCoef.cpp at line 185 ... ERROR MESSAGE: this statement should not be reached!` (slide 10). The message names the pressure class, not the velocity block you edited; $a = -1$ or $b = -1$ fails the same way
a structure point on or outside the box (chapter 6's tests)	stops at start-up or at the first regrid with a point-outside-domain error
Windows line endings in the input file	stops: `Illegal character token in input` (chapter 3)

The muParser errors print the whole offending expression under `in:`, so they are easy to find. "Position" counts characters from 0 in that string.

WHAT CAN GO WRONG: RUNS ANYWAY (TESTED)

The more dangerous cases: the run starts, and the mistake shows only in the result, or in the log file.

The edit	What happens
a misspelled key: `gcoef_functoin_1` for the outflow side	runs. The log *file* (`IB2d.log`, not the screen) has: `WARNING MESSAGE: muParserRobinBcCoefs::muParserRobinBcCoefs(): no function corresponding to key ``gcoef_function_1'' found for side = 1; using gcoef = 0.0.` With $a = 1$ that makes the outlet a wall: inflow on the left into a closed box. The same warning, with `acoef = 0.0`, is what a 2D block pasted into a 3D input produces for sides 4 and 5
`periodic_dimension = 1, 0` with the inflow still written on sides 0 and 1	runs, no warning; the inflow is ignored and the box is periodic in x with sliding top and bottom (slide 45)
a traction condition on the outlet ($a = 0$, $b = 1$, $g = 0$ for both components)	runs, no warning; a valid open outlet (slides 13, 42). Remember `normalize_pressure = FALSE`

> Always read the log file after the first short run. The warnings go to the log, not to the terminal, and the first one is on line 28 of a file that will have thousands.

WHAT CAN GO WRONG: THE PHYSICS (1)

Runs that are correct input files and still wrong.

- An impulsive start: $g = U$ at $t = 0$ with the fluid at rest. Pressure spike, a kick to the structure, often `Time step size change encountered` at step 1 with `error_on_dt_change`. Ramp (slide 28), or set a matching initial condition (slide 48).
- Net flux that does not balance with every side prescribed (slide 12): the projection step still runs and the divergence is pushed to the boundary cells; the pressure and the flow near the sides are meaningless. Open a side or fix the profiles.
- Backflow at an open outlet: a wake eddy reaching the outlet brings fluid in through a side with $p = 0$; the run can blow up there. Lengthen the box, or use the stabilizer (slide 44).
- A wake squeezed by a prescribed outlet: no instability, but a wrong flow. Compare with a longer box (slide 33).

WHAT CAN GO WRONG: THE PHYSICS (2)

- Periodic images too close: with `periodic_dimension = 1` the structure sees its own copies at a distance `x_up - x_lo`; the glycocalyx is *meant* to be an array, the leaf is not (slide 46).
- The structure in the wall's boundary layer: a no-slip wall two cells from a structure changes its forces; IB/08's ring of target points on the floor is the measured case (the force grows linearly with time, README).
- A coarsest level too coarse to see the structure: $M = 2$ cells across and a structure in one of them; the levels are placed at coarse-cell granularity (slide 56).
- Cells that are not square: `x_up - x_lo` changed without `domain_boxes`, or the other way round. The run proceeds; the kernel and the structure spacing are wrong in one direction, and nothing says so (slide 50).
- Too few frames, or too many: an interval in steps copied from a run with a different `DT` gives three frames, or three thousand (slide 63).
- The wrong side: a lid written on side 3 in a 3D input is a wall in y, not the top; the top is side 5 (slide 18).

HOMEWORK

From the 2015–2020 courses and the READMEs (Appendix C collects the full set):

1. IB/03: make the top and bottom walls stick (`gcoef_function_2` and `_3` of the x-block to `"0.0"`) and compare the wake; then bring the flow in from the top instead of the left (which blocks change?).
2. IB/02: double `f`, halve `d`, and make the flow reverse. Describe the profile at a peak and at a zero crossing.
3. IB/03: put the leaf in a $2L \times L$ box with the leaf a quarter of the way in. Write the new `N`, `domain_boxes`, `x_lo`, `x_up` and patch sizes before you run; check `DX` is unchanged.
4. IB/03: make the outlet open (traction, $g = 0$ on both components) with `NORMALIZE_PRESSURE = FALSE`, and compare the wake near the outlet with the prescribed-outlet run.

> What to hand in (the 2018–2020 pattern): the edited blocks, one VisIt frame per case with the boundary velocity visible, and two sentences on what changed. For item 3, also the arithmetic of slide 56.

Extension: in IB/08, set `SHEAR_START = 1` and compare the force on the fibre at $t = 0.2$ s with the ramped run (the README has both numbers).

RULES OF THUMB

- Every side, every component, three keys; $a = 1$ for a velocity, $b = 1$ for a stress, never both and never -1.
- Constants used in the quotes are defined *in the block*; top-level names are invisible there.
- `cond ? a : b`, not `if(cond, a, b)`.
- Ramp the inflow with `tanh(2*t)`; start the fluid at rest.
- Prescribed velocity on all sides: fluxes must balance. Any open side: `normalize_pressure = FALSE`, and think about backflow.
- Periodic overrides the blocks, silently.
- Change `x_lo`/`x_up` and `domain_boxes` together; keep `DX` equal in all directions; coarsest counts ≥ 4 and divisible by the smallest patch; finest counts divisible by $\mathrm{REF_RATIO}^{\mathrm{MAX_LEVELS}-1}$.
- Keep the structure at least a point spacing (better: a few cells) off every side, for the whole motion.
- Intervals are in steps; fifty frames per run is a good default.
- Two steps, read the log file, one frame in VisIt, then the real run.

SUMMARY

1. A boundary condition in IBAMR is $a\,u + b\,\partial u/\partial n = g$, given per side and per velocity component as three expressions. The fluid solver takes only $(1, 0)$, a velocity condition, and $(0, 1)$, a traction condition; for the normal component the traction condition sets the pressure, so an outlet pressure is $g = -p$ with $b = 1$.
2. Sides are numbered 0–5 as $2 \times$ axis $+$ (lower 0, upper 1). The expressions may use `t`, `X_0`, `X_1`, `X_2`, the built-in constants, the block's own constants, muParser's functions, and `?` `:` for conditions.
3. The examples: a ramped uniform stream on all sides (IB/03, IBFE/10), a pulsing parabola on two sides (IB/02), periodic sides with a floor and a sliding lid (IB/08), a scalar with Dirichlet sides (IBFE/12), and upstream an inflow with an open outlet, free-slip walls and a stabilizer.
4. The domain is the coarsest index box plus two physical corners; the finest grid is the coarsest times $\mathrm{REF_RATIO}^{\mathrm{MAX_LEVELS}-1}$; keep the cells square, the coarsest counts ≥ 4, the patches fitting, the structure off the edges, the intervals in steps.
5. The input errors stop the run with a quoted expression or an unhelpful "should not be reached"; the misspelled key and the periodic override do not stop it. Read the log.

Next, chapter 8a: the physical parameters at the top of the file, and how to change them without breaking the run; then 8b, non-dimensional analysis.

WHAT CHANGED SINCE THE OLDER COURSE NOTES (1)

- The Neumann slide's "$b = -1, 1$" is explained and corrected for the input file ($b = 1$ on both sides); the fact that the fluid solver accepts only $(1, 0)$ and $(0, 1)$, and what each pair means for normal and tangential components, is stated for the first time, from the source.
- The traction (open-side) condition, `TRACTION` versus `PSEUDO_TRACTION`, the derived pressure condition, the "$g = -p$" recipe and `normalize_pressure` are new; the 2013 "you cannot specify the pressure" is kept as history with Boyce's source/sink hints.
- The in-line `if()` is replaced by the ternary, with the actual error message; IB/02's input is the fixed version.
- The variable and constant list is the full one from `muParserRobinBcCoefs.cpp`, and the block-constant trap is tested and named.
- The open-boundary stabilizer and the upstream stabilized-cylinder input (with its free-slip walls and shedding nudge) are new material.

WHAT CHANGED SINCE THE OLDER COURSE NOTES (2)

- Part II keeps the 2017/2020 worked example (the glycocalyx column) and adds the flow-tank example, the reasons behind the divisibility and minimum-size rules, the level and patch-size tables across the examples, the tagging and regridding keys, and the 2015 output and Δt cautions with the examples' actual values.
- "What can go wrong" is tested: six edits run, with the messages quoted, including the misspelled-key warning that only the log file shows and the periodic override that shows nothing at all.
- The pulsatile channel is shown at a peak and at a trough of its pulse, with the annular effect explained; the leaf is shown in its box with the boundary vectors visible.
- The example names are the series' (IB/02, IB/03, IB/08, IBFE/10, IBFE/12) with the old names noted; the UNC and Drive download links of the 2013–2017 slides are gone, as is the 2020 slide on editing the UNC makefiles for the UA cluster (chapter 3 covers CMake builds instead).

REFERENCES

- IBAMR source, version 0.19.0: `src/navier_stokes/INSStaggeredVelocityBcCoef.cpp`, `INSStaggeredPressureBcCoef.cpp`, `INSProjectionBcCoef.cpp`, `INSHierarchyIntegrator.cpp` (the `traction_bc_type`, `normalize_pressure`, vorticity-tagging keys), `StaggeredStokesOpenBoundaryStabilizer.cpp`; `ibtk/src/boundary/physical_boundary/muParserRobinBcCoefs.cpp`; `ibtk/src/utilities/muParserCartGridFunction.cpp`, `AppInitializer.cpp`; `ibtk/contrib/muparser`; examples `IB/explicit/ex2` (`input2d.cylinder_stabilized`, `example.cpp`), `IIM/ex9`, `navier_stokes/ex0`. https://github.com/IBAMR/IBAMR
- muParser documentation: https://beltoforion.de/en/muparser/
- IBAMR FAQ: https://github.com/IBAMR/IBAMR/wiki/FAQ; users' group: https://groups.google.com/g/ibamr-users. The 2013 slide also pointed to Boyce Griffith's then page at NYU (`math.nyu.edu/~griffith`) for a pulsing-pressure example; his group is now at UNC.
- SAMRAI (bundled as IBSAMRAI2): the `CartesianGridGeometry` and `GriddingAlgorithm` input keys; the comments in the examples' input files are the working reference.
- The examples of this series: IB/02 (pulsatile channel), IB/03 (tethered leaf), IB/08 (glycocalyx), IBFE/10 (3D flow tank), IBFE/12 (advection–diffusion), each with a README that states what its boundary conditions do and what was measured.
- The tests behind slides 66–67: `wsl/final/ch7_errors.sh` in the working folder of these notes; the figures: `wsl/final/visit_ch7.sh`, `visit_ch7b.sh`, `run_ib02_short.sh`.
- Sources: *IBAMR boundary conditions* (MATH 769, February 5, 2013); *2 – IBAMR Tutorial: Boundary Conditions* (2013, edited 2017; the pressure notes); *MATH 769 Lecture 7: Boundary Conditions and How to start changing things* (2015); *4 – IBAMR Tutorial: Boundary Conditions* (2017); *Using different domain dimensions* (2017); *Lecture 4 – IBAMR Tutorial: Boundary Conditions* (2020, Parts I and II).